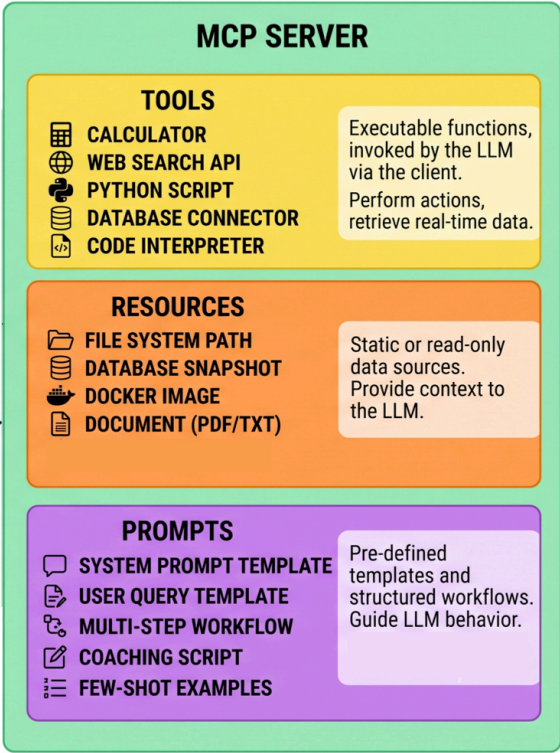
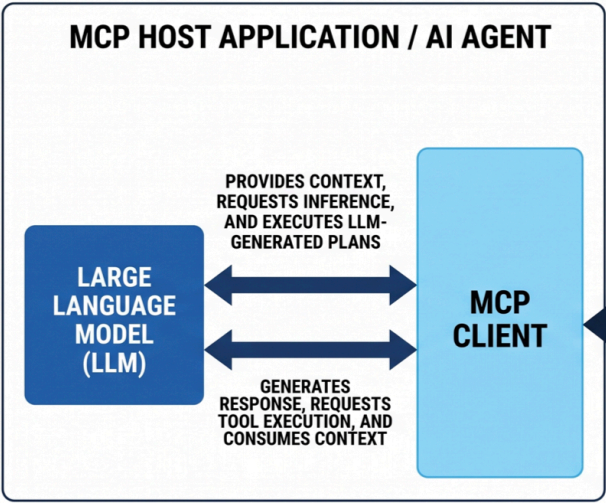
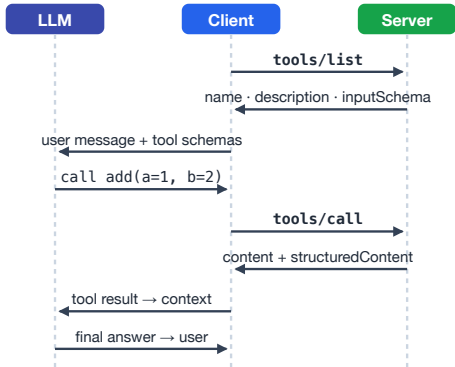


Model Context Protocol (MCP)



DISCOVER → CALL LOOP (tools/list)

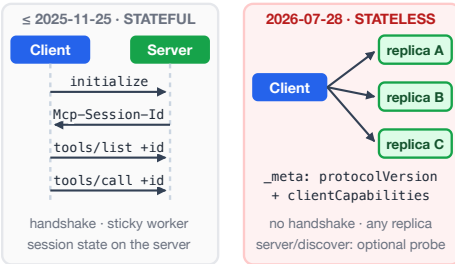


Dynamic discovery: the client adapts to whatever each server exposes, no hardcoded knowledge. Re-list on `list_changed`. Tools should come back in a stable order (prompt-cache friendly).

WHAT MCP IS NOT

- REST API** MCP adds discovery, typed schemas the model can read, and client-agnostic reuse.
- RAG** RAG retrieves text into context; MCP grants capabilities. Complementary: resources are often the RAG surface.
- function calling** That is per-vendor. MCP is vendor-agnostic: one server, every host.

SESSION → STATELESS (2026-07-28)



The 2026-07-28 Model Context Protocol (MCP) specification fundamentally refactored the protocol architecture to be entirely stateless, completely eliminating the protocol-level initialization handshake and the traditional `Mcp-Session-Id` header. This update shifted MCP from a connection-oriented model into a stateless, request-response architecture optimized for cloud-native web infrastructure, microservices, and serverless architectures.

SERVER IN PYTHON · `pip install "mcp[cli]" → 2.x`

```
from mcp.server import MCPServer

mcp = MCPServer("Demo", instructions="Prefer add.")

@mcp.tool()
def add(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b

if __name__ == "__main__":
    mcp.run()
```

- Name, description and JSON Schema come from the function name, docstring and type hints. Nothing is declared twice.
- Sync `def` runs on a worker thread; `async def` on the event loop.

TRANSPORTS

- stdio** Default, local. Host spawns your file; stdin/stdout is the wire. Never `print()` to stdout, log to stderr.
- Streamable HTTP** Anything you deploy. `POST /mcp`; reply is one JSON body or an SSE stream.

RUN IT · transport options live on `run()`, never on `MCPServer()`

```
mcp.run()
mcp.run(transport="streamable-http",
        host="0.0.0.0", port=8000)
# also: streamable_http_path, json_response=True,
#       stateless_http=True (only affects 2025 clients)
```

Keep `run()` under `__main__`: `mcp dev/run/install` and tests import the file. HTTP defaults to 127.0.0.1:8000 with DNS-rebinding protection (pass `transport_security=` behind a real hostname).

PLUG INTO A HOST · one launch command, configure today

```
{
  "mcpServers": {
    "demo": {
      "command": "uv",
      "args": ["run", "--with", "mcp[cli]",
              "mcp", "run", "/abs/path/server.py"]
    }
  }
}
```

Claude Desktop (`mcp install` writes it), Cursor `.cursor/mcp.json`, Kiro `.kiro/settings/mcp.json`. VS Code uses `servers + "type":"stdio"`. Remote server: `"url": "https://.../mcp"`. Absolute paths, then restart the host.

JSON-RPC 2.0 METHODS

<code>server/discover</code>	protocol versions · capabilities · identity (optional probe)
<code>tools/list</code>	discover tools
<code>tools/call</code>	execute a specific tool
<code>resources/list</code>	discover resources
<code>resources/templates/list</code>	dynamic, parameterized data sources
<code>resources/read</code>	URI → content
<code>prompts/list</code>	reusable prompt templates
<code>prompts/get</code>	name, args → text/structured content
<code>completion/complete</code>	argument autocomplete
<code>subscriptions/listen</code>	open a long-lived, server-to-client notification stream

```
→ {"jsonrpc":"2.0",
  "id":1,
  "method":"tools/call",
  "params":{"name":"add",
    "arguments":{"a":1,"b":2},
    "_meta":{"protocolVersion":"2026-07-28",
      ".../clientCapabilities":{}}}
}

← {
  "jsonrpc":"2.0",
  "id":1,
  "result":{"resultType":"complete",
    "content":[{"type":"text","text":"3"}],
    "structuredContent":{"result":3}}
}
```

SPEC 2026-07-28 · what changed vs 2025-11-25

- Stateless core.** No `initialize` handshake, no `Mcp-Session-Id`; each request carries protocol version + client capabilities in `_meta`, so any replica can answer. Cross-call state = explicit handles in tool args.
- Servers can't call clients.** Roots/sampling/elicitation push → Multi Round-Trip Requests: a result with `resultType:"input_required"`, client retries with `inputResponses`. Python: `Annotated[... , Resolve(ask)]` returning `Elicit(...)`.
- List/read results carry `ttlMs + cacheScope` (clients cache); HTTP POSTs carry `Mcp-Method/Mcp-Name` for header-only routing.
- SDK 2.x serves 2025 and 2026 clients from the same app; `Client` probes and falls back automatically.